

Document Number:	P0804R0
Authors:	Tom Honermann
Date:	2017-10-15
Audience:	Evolution Working Group
Reply-to:	Tom Honermann <tom@honermann.net>

Impact of the modules TS on the C++ tools ecosystem

- [Introduction](#)
- [Integration strategies](#)
 - [Consume a module artifact from another provider](#)
 - [Translate a module artifact from another provider](#)
 - [Build observation](#)
 - [Build system duplication](#)
 - [Prohibit use of module features](#)
- [Acknowledgements](#)
- [References](#)

Introduction

This paper provides a detailed description of the motivation for the concern expressed in comment US/001 of the PDTS 21544 collated comments document [\[PDTS comments\]](#).

Consider the following program that utilizes features from the [Modules TS \[N4667\]](#) and a modularized standard library implemented as specified by [P0581R0 \[P0581R0\]](#). The code defines a module (foo) that utilizes entities imported from a dependent standard library module (std.core) and a user provided module (bar).

foo.cpp

```
module foo;

import std.core;
import bar;

export std::vector<widget> get_some_widgets() {
    /* ... */
}
```

bar.cpp

```
module bar;

export class widget { /* ... */};
```

The following build commands exemplify those required to build the example using Microsoft's implementation of the Modules TS. Note that the order of the invocations is significant; the module interface unit of module bar (bar.cpp) must be compiled before the module interface unit of module foo (foo.cpp) as the latter requires the existence of the module artifact (bar.ifc) produced by the compilation of bar.cpp.

Build commands

```
rem Compiling bar.cpp produces bar.ifc and bar.obj.
cl /EHsc /MD /std:c++latest /c bar.cpp /experimental:module /module:interface
rem Compiling foo.cpp produces foo.ifc and foo.obj.
cl /EHsc /MD /std:c++latest /c foo.cpp /experimental:module /module:interface /module:reference bar.ifc
```

Now consider what is necessary for a C++ tool to consume the source code above. Historically, C++ tools have required some amount of configuration to handle preprocessor state; macro definitions and header search paths for example. Once configured,

access to the source code (and potentially, support for some language extensions) is all that is needed to successfully parse the source code.

The Modules TS adds additional requirements not previously needed in order to successfully parse source code. With an example like the one above, access to the source code and preprocessor configuration alone no longer suffices. The tool must now be able to resolve module import declarations to either the source code for the corresponding module interface unit, or to some module artifact that provides the exported entities for the module.

Assuming an implementation strategy similar to Microsoft's, the following capabilities and information must be available either directly in the tool or in a script or build system that invokes the tool:

- The ability to produce and consume module artifacts containing module export information.
- The transitive set of all dependent modules for each translation unit to be processed, the source code for each module interface unit and its included headers, the preprocessor options and language dialect options necessary to compile each module interface unit, and an order in which to build each module interface unit so that dependent module artifacts are available as needed.
- The ability to map a module import declaration to a module artifact.

This information may be difficult and costly to obtain and maintain, particularly when deep module hierarchies are present or when module dependencies are platform dependent (consider a standard library implementation that imports implementation specific modules in the source code for its public facing module interface units).

Though module artifact are not intended to be a distribution format or an alternative to access to source code, motivation exists to use them in this way. At least one large company is in the process of modularizing its source code (not currently using the Modules TS) in order to reduce overhead in its distributed build system by distributing module artifacts in lieu of source code. This is clearly problematic for tool providers since, without access to the source code, compilation is impossible.

Many C++ tools do not supply their own standard library. Other tools, such as static analyzers, realize their maximum value when they are able to make use of the standard library of a particular compiler provider. Source code is often dependent upon a particular implementation of the standard library and can't be successfully parsed with other implementations. For these reasons, many tools are dependent on interoperability with standard libraries provided by other providers; providing their own modularized standard library is not a feasible option.

The remainder of this paper looks at potential options for integrating a tool with a modularized code base.

Integration strategies

Consume a module artifact from another provider

Module artifact formats (e.g., the `bar.ifc` and `foo.ifc` files above) fall outside the scope of the Modules TS; they are implementation details if they exist at all. Microsoft is currently using the [open source IPR project](#) ^[IPR] which, in theory, could allow tools to consume Microsoft's module artifact files.

The effort required to consume a module artifact and translate the information contained within it to a given tool's internal data representation may be prohibitively expensive. A module artifact must be able to represent nearly the entirety of the C++ language; translating that information to another form is likely to require a significant effort. For some tools, a simpler approach might be to examine the module artifact to determine what source code was used to construct it (assuming the module artifact retains this information; something that seems likely as it would be useful in generating diagnostics for consumers), and then translate that source code directly (assuming the source code is present). These tools might also benefit if the module artifact preserves the command line options used to compile the module interface unit source code.

Microsoft does make the source code to its standard library implementation available, so, in theory, a tool could construct its own module artifacts from it. As of the Visual Studio 2017 release, Microsoft does not provide distinct source code for the module interface units distributed with the product, nor is a build system for compiling the standard library provided. The (experimental) `.ifc` files distributed with the product were constructed using the `/module:name` and `/module:export` options as described in the "Consuming Existing Legacy Header Files as Module Interfaces" section of [a blog post introducing the modules features implemented in Visual Studio 2015 Update 1](#) ^[Modules in VS2015U1]. Assuming Microsoft continues with this strategy for producing standard library modules, tools that require module interface unit source code will also have to support a translation mode matching the behavior of the `/module:name` and `/module:export` options.

When asked about the potential for tools to consume Microsoft generated module artifacts, Gabriel Dos Reis provided the following statement on behalf of Microsoft.

We thank you for sharing an early draft of your paper. Tooling, especially semantics-aware development tools, that supports sound and scalable C++ software architecture practice is at the heart of Microsoft's efforts behind the C++ modules proposal. Microsoft is a producer and a huge consumer of C++ technologies and tools, including third party and open source components. Microsoft is fully committed to opening up its compiled module interface binary format and tooling regarding C++ modules, and encourages the larger C++ community to coalesce around shared formats or library APIs or conversion protocols. Microsoft will address the specific points raised in this draft document in a separate WG21 document. Modules represent an unprecedented opportunity for the C++ community to up its game regarding tooling, and we hope the community seizes it for the greater good of all of us.

It isn't yet known what module artifact formats other implementors may choose, but it seems unlikely that all implementors will converge on a single portable format. In particular, it seems likely that Clang will use its existing, optimized, non-portable format currently used for pre-compiled headers and [Clang's own module system](#) [\[Clang Modules\]](#). GCC appears to be forging ahead with its own module artifact format as indicated on the [GCC wiki](#) [\[GCC Modules\]](#). Thus, even if it turns out that consuming Microsoft's module artifact files is a feasible solution, that solution is unlikely to be feasible for all compiler providers.

Requirements:

- Access to the module interface unit source code *may* be required.
- Module artifact formats must be public, stable, extensible, etc...

Translate a module artifact from another provider

Similar to direct consumption, another approach would be to translate a module artifact from another provider, for example, to generate source code that could be compiled to produce a module artifact in another format.

If compiler providers were to provide a tool capable of such generation, it would obviate the need for each tool provider to be able to directly consume module artifacts from every compiler provider. Ideally, compiler providers would collaborate to define a common interface each could implement.

Requirements:

- Access to the module interface unit source code is *not* required.
- Tools must be available to consume a module artifact and generate (portable standards compliant with exceptions for exported entities that use language extensions) source code suitable for compilation as a module interface unit.

Build observation

It is possible for a tool to discover modules by observing a build system in action. Upon invocation of a recognized compiler, the command line for the invocation could be analyzed, its mode (preprocess-only, translate, produce a pre-compiled header, produce a module artifact, etc...) determined, and actions taken to replicate that action as necessary.

For example, for the build commands provided above, a tool could observe the first invocation (for `bar.cpp`), discover that the command line includes `/module:interface`, and then use other information provided on the command line to determine how to compile `bar.cpp` so as to build its own module artifact for later consumption. When the second invocation (for `foo.cpp`) is then observed, pending any errors, the dependent module artifacts will be available to facilitate translation (and production of an additional module artifact for importers of module `foo`).

This is a technically challenging, non-portable solution that requires deep platform-specific integration and compiler-specific support.

This approach fails when the compilation of needed module interface units is not observed.

Requirements:

- Access to module interface unit source code.
- The ability to recognize and understand compiler invocations.
- Module artifact generation must be observed during the build (This is unlikely to be the case for standard library modules)

- In a distributed build environment, requires any module artifacts produced by the tool to also be distributed (or rebuilt).

Build system duplication

Assuming a tool has support for producing and consuming module artifacts, it would be possible to script invoking the tool for each needed module. This likely amounts to duplicating a portion of an existing build system that already supports one or more compilers or tools.

Requirements:

- Access to module interface unit source code, including those for standard library modules.
- The preprocessor options, language dialect options, etc... needed to translate each module interface unit.
- In a distributed build environment, requires any module artifacts produced by the tool to also be distributed (or rebuilt).

Prohibit use of module features

This is the status quo. Tool providers can elect not to support modularized source code.

This may not be such a confrontational stance to take. Some tools only require access to a portion of a code base. For example common uses of [SWIG](#) ^{[[SWIG](#)]} require just a few translation units and their headers. Frequently, the preprocessor can be used to further restrict necessary dependencies. It is conceivable that conditional compilation could suffice for many use cases. However, such a stance would likely become increasingly difficult to maintain; especially as pressure increases to modularize the lowest level of dependencies, such as the standard library.

Support for a partially modularized code base would likely require resolution of comment US/002 of the PDTS 21544 collated comments document [\[PDTS comments\]](#) to ensure compatibility between translation units that access the same entities via module exports vs declarations in the global module.

Requirements:

- A non-modularized source code base.

Conclusions

These challenges for tool providers pose a problem for the adoption of modules as specified by the Modules TS. Likewise, if adoption of the Modules TS is high, the difficulty in providing support for modularized code bases could result in a chilling effect on the development of new tools unless convenient methods are found to address the integration concerns described here.

It isn't clear how, or if, these challenges can be addressed within the Modules TS given that build systems and implementation strategies can not realistically be prescribed. As a result, this paper offers no particular solutions for the concerns it raises.

It is clear that implementation choices will impact the degree of difficulty tool providers and tool users will face supporting modularized code bases; particularly for portable code compiled by multiple compilers. This difficulty will be significantly reduced if implementors work together to establish common utilities and/or protocols that enable a uniform abstract interface tools can use to resolve import declarations.

Acknowledgements

Thank you to the good people at Synopsys who funded, reviewed, and helped to refine this paper. In particular, Charles-Henri Gros, Thierry Lavoie, Michael Price, Tim Prince, and Tyler Sims.

Special thanks to Billy O'Neal, Gabriel Dos Reis, and Richard Smith for reviewing this paper, providing feedback and clarifications, and continuing to engage in design discussions.

References

[PDTS_comments]	ISO_IEC PDTS 21544 - JTC001-SC22-N5233 Collated Comments
[N4667]	Working Draft, Extensions to C++ for Modules http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/n4667.pdf
[P0581R0]	Standard Library Modules http://www.open-std.org/JTC1/SC22/WG21/docs/papers/2017/p0581r0.pdf
[Clang_Modules]	Clang Documentation: Modules https://clang.llvm.org/docs/Modules.html
[GCC_Modules]	GCC Wiki: C++ Modules https://gcc.gnu.org/wiki/cxx-modules
[Modules_in_VS2015U1]	C++ Modules in VS 2015 Update 1 https://blogs.msdn.microsoft.com/vcblog/2015/12/03/c-modules-in-vs-2015-update-1
[IPR]	Compiler-neutral Internal Program Representation for C++ https://github.com/GabrielDosReis/ipr
[SWIG]	Simplified Wrapper and Interface Generator http://www.swig.org